Shoolini University

Progress Report 1 – Initial Stage

Submission Date: 10 November 2026

---

1. Student Details

Name	Md Faiz Alam
Course	BCA (Bachelor of Computer Applications)
Semester	6th Semester
Admission No.	UGO24156735
Project Title	StockFlow – Inventory Management System for Small and Medium Businesses

2. Project Objective

StockFlow is a web-based inventory management system for small and medium businesses that still track stock in registers or spreadsheets.

It will keep an exact, real-time record of every product, every unit received and every unit issued, with who made the change and why.

On top of this record it will manage suppliers, purchase orders, customers and sales orders, and give managers reports and a dashboard that show what is running low and needs to be reordered.

3. Proposed Features

- **Secure login** with role-based access, so each user (Admin, Manager, Staff) only sees and does what their role allows.
- **Product catalogue** with a unique SKU, category, unit price and reorder level for every item; products are archived instead of deleted so history is never lost.
- **Stock control:** receive and issue stock with a reason, keep a running balance, block negative stock and ignore accidental double submissions.
- **Movement history:** a permanent log of every stock change with date, user, quantity and balance before and after.
- **Supplier management** with contact details and the products each supplier provides.
- **Purchase orders** to suppliers, including deliveries that arrive in parts.

- **Customers and sales orders,** where fulfilling an order automatically issues the stock.
- **Low-stock and out-of-stock alerts** based on each product's reorder level.
- **Reports** with search and date filters, downloadable as CSV and PDF.
- **Analytics dashboard** with key figures and charts of stock movement and stock value.
- **Activity log** recording important user actions for accountability.
- **Responsive interface** that works on desktop and mobile, with light and dark themes.
- **Online deployment** on free cloud hosting so the system can be used from anywhere.

4. Technology Stack

Category	Technology	Purpose
Programming language	Python 3.11	Server-side business logic
Web framework	Django 5.2	Pages, URL routing, forms, ORM, authentication and admin panel
Database	PostgreSQL 17	Relational storage; row-level locking keeps stock updates correct
Frontend	HTML5, CSS3, JavaScript (Django templates)	User interface and responsive layout
Dynamic UI (planned)	HTMX, Chart.js	Live search without page reloads; dashboard charts
Local environment	Docker Desktop + Docker Compose	Runs the PostgreSQL server on the development machine
Configuration	python-dotenv (.env file)	Keeps passwords and secret keys out of the source code
Testing	Django test framework	Automated tests for every module
Version control	Git + GitHub	Source code history and backup
Hosting (planned)	Render (web app) + Neon (PostgreSQL)	Free online deployment
Editor	Visual Studio Code	Development and debugging

5. System Design

The system follows Django's three-layer structure: a **presentation layer** (HTML templates and CSS), an application layer (Django views, forms and service functions) and a **data layer** (PostgreSQL accessed through Django models). The code is divided into separate modules (Django apps), each responsible for one area:

- **Accounts module:** custom user model, sign-in and sign-out, and permission checks that decide which pages and actions each user can access.
- **Inventory module:** products, current stock balances and stock movements. All stock changes will go through one service function that locks the product row, checks the rules (enough stock, valid quantity, user permission) and records the movement.
- **Supplier module (planned):** supplier contacts and supplier–product links.
- **Purchasing and Sales modules (planned):** purchase orders, customers and sales orders that call the same stock service.
- **Reports and Dashboard module (planned):** read-only summaries, charts and exports.

Planned database tables

Table	Stores
users	Login accounts and their permissions
products	SKU, name, category, price, reorder level, active/archived
stock_balances	Current quantity on hand for each product
stock_movements	Every receipt and issue with user, reason and balances
suppliers	Supplier contact details and linked products
purchase_orders	Supplier contact details and linked products
customers, sales_orders, sales_order_lines	Customer details and what they ordered

Basic flow

A user signs in → opens the product catalogue → records a stock receipt or issue → the stock service checks the permission and quantity, locks the product, updates the balance and saves a movement record → the updated stock appears on the dashboard and in reports.

6. Work Done So Far

- Project idea finalised and problem statement defined (small businesses losing track of stock kept in registers and spreadsheets).
- Requirements and module list prepared; technology stack and system structure decided.
- GitHub repository created with README, .gitignore and line-ending rules.
- Django project set up in a Python virtual environment, with settings read from a .env file so secret keys and passwords are never stored in the code.
- **Custom user model** created at the very start, so roles and extra user details can be added later without rebuilding the database.
- **Sign-in page** implemented with CSRF protection; sign-out works only through a secure POST request.
- **Responsive workspace layout** built: sidebar navigation, top bar, overview page, and a light/dark theme switch.
- **Product catalogue module** completed: add, edit, search, filter by category, and archive/restore products, with unique SKUs and permission checks on every page.
- Development database moved from SQLite to **PostgreSQL 17,** running locally with Docker Compose; database migrations applied.
- **32 automated tests** written for sign-in, sign-out, page access and the product catalogue; all passing.

Next steps

- Build the stock engine (receive and issue stock) with movement history.
- Add the supplier module and set up automatic testing on GitHub.
- Start the reports module.